

# PROJECT-BASED LEARNING



## PHASE-I EVALUATION

### **Data-Structure-Based USB Malware Activity Monitor Using C and C++ OOP**

Team ID: DSCPP-III-2026-T370

**Department of Computer Science & Engineering**  
Graphic Era (Deemed to be University), Dehradun  
Academic Session 2026-27

# Team & Mentor Details



## Team Information

**Team ID:** DSCPP-III-2026-T370  
**Domain:** *Cybersecurity — USB Malware Detection*  
**Approach:** *Data structures in C, wrapped in C++ OOP*

## Team Members

- ① *Divyanshu Panwar (Team Lead) – 2510370085*
- ② *Abhay Rai – 2510370531*

## Mentor

*Siddharth Thapliyal*

**Interactions completed:** \_\_\_\_\_

## Project Title

*“Data-Structure-Based USB Malware Activity Monitor Using C and C++ OOP”*



## Problem Statement

*Traditional, signature-based antivirus tools can miss zero-day exploits or malicious activity triggered the moment a USB device is inserted. Existing solutions (Defender, USBGuard, hardware firewalls) are heavy, closed-source, and give no visibility into why something was flagged.*

## Motivation

- *USB drives remain a top attack vector, especially in air-gapped or network-restricted environments.*
- *Build a lightweight, transparent alternative to closed-source, black-box tools.*
- *Show exactly which data structure flags a threat and why — not just an alert.*
- *Prototype relies on simulated device/file event logs, not live OS hooks.*

## Target Users / Stakeholders

- *System admins in high-security or air-gapped environments.*
- *Users who want transparent, explainable USB threat detection.*
- *Students/researchers studying data-structure-driven security tools.*
- *Future systems that could integrate real OS-level USB hooks.*



## Project Objectives

- ① *Build a working USB device and file-activity monitor in C/C++ using self-implemented data structures.*
- ② *Detect device insertion/removal in real time and log events using arrays or linked lists.*
- ③ *Build a hash table mapping device IDs (VID:PID) and known-malicious file hashes for  $O(1)$  lookups.*
- ④ *Add a FIFO event queue to process device and file-system events as they occur.*
- ⑤ *Implement a stack-based quarantine to undo/rollback the last  $N$  flagged operations.*
- ⑥ *Organize scanned files by type using a binary search tree (or trie) for fast range/prefix search.*

**Key Objective:** *Detect and prioritize suspicious USB activity in a maintainable C++ OOP structure (USBDevice, FileEvent, AlertManager, Logger).*



## Proposed Approach

- *Write core data structures in C for low-level control and performance.*
- *Wrap them in a C++ OOP layer for structure and usability.*
- *Device Registry: self-built hash table (VID:PID  $\rightarrow$  trust status).*
- *Event Queue: FIFO pipeline for device/file-system events.*
- *Quarantine Stack: undo/rollback of flagged files.*

## Key Features

- *Hash-based device & signature lookup ( $O(1)$ )*
- *FIFO event-processing pipeline*
- *Stack-based quarantine and rollback*
- *BST / trie file categorization by type*
- *Quicksort / mergesort based activity reports*

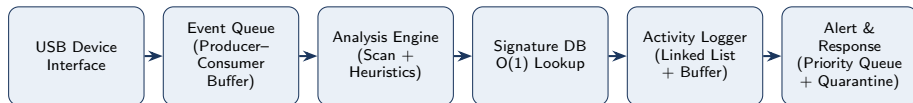
# Integration of PBL Course Concepts



| Course               | Concepts Applied                                                            | Project Module                                                |
|----------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------|
| Data Structures in C | Hash table, queue, stack, BST/trie, linked list                             | Device registry, event queue, quarantine, file categorization |
| OOPs with C++        | Class hierarchy (USBDevice, FileEvent, AlertManager, Logger); encapsulation | Maintainable, structured detection system                     |
| Algorithms           | Quicksort / mergesort; $O(1)$ average hashing                               | Activity reporting & fast signature lookup                    |

**Requirement:** combine self-implemented data structures, algorithms, and C++ OOP in one working USB monitor — **SentinelUSB**.

# System Architecture / Workflow



## Major Components

- *Device Registry (hash table)*
- *Event Queue (FIFO)*
- *Analysis Engine & heuristics*
- *Signature DB / Activity Logger*
- *Alert & quarantine module*

## Data / Control Flow

- *Device/file events enter the FIFO queue as they occur.*
- *The analysis engine checks events against the signature DB.*
- *Flagged activity is logged and raised through the priority-queue alert & quarantine module.*

# Technology Stack & Methodology



## Technology Stack

- **Language:** *C (data structures) + C++ (OOP layer)*
- **Dev environment:** *VS Code / Code::Blocks, GCC/G++*
- **Data structures:** *hash table, queue, stack, BST/trie, linked list*
- **Algorithms:** *quicksort / mergesort, hashing*
- **Version control:** *Git / GitHub*
- **Stretch goal:** *Qt or Flask+REST dashboard*

## Development Methodology

- ① Requirement and problem analysis
- ② Core class/struct design (USBDevice, FileEvent, Alert)
- ③ Data-structure implementation (hash table, queue, stack, tree)
- ④ C++ OOP integration and testing with simulated CSV event logs

# Initial Progress & Team Contribution



## Progress Achieved

- *Project proposal, motivation, and current-solution gap analysis defined.*
- *Goals and milestones identified.*
- *Core data structures and C++ OOP class design selected.*

## Individual Contribution

| Member           | Role        | Contribution                                                  |
|------------------|-------------|---------------------------------------------------------------|
| Divyanshu Panwar | Team Lead   | Team leadership and project coordination, and data collection |
| Abhay Rai        | Team Member | Report formation                                              |

# Roadmap, Expected Outcomes & References



## Roadmap

- 1 Phase-I: Proposal & Design
- 2 Phase-II: Development
- 3 Phase-III: Final Implementation

## Expected Outcomes

- Console app “SentinelUSB” catching real-time USB insertion/removal
- Hash-based whitelist/blacklist file-operation flagging
- Stack-based quarantine and rollback of flagged files
- Activity reports: most active devices, alert-type breakdown

## References

- 1 *GeeksforGeeks — data structure implementation guides*
- 2 *Deitel & Deitel, C++ How to Program*
- 3 *USBGuard project documentation*
- 4 *NIST SP 800-83 — Guide to Malware Incident Prevention and Handling*

## Deliverables

- Full source code on GitHub
- Sample simulated event-log CSV used for testing
- Report explaining design decisions and data-structure choices
- Live demo: device monitoring, event processing, reporting

**Thank You**

Questions & Suggestions

*Data-Structure-Based USB Malware Activity Monitor Using C and C++  
OOP*

**Team DSCPP-III-2026-T370**

Divyanshu Panwar & Abhay Rai